

Reviewer Pack — Minimal Quadratic Residues and Communication Complexity Grow...

Entry 2b8c15d516fe · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 00:37:05 UTC

Minimal Quadratic Residues and Communication Complexity Growth in Planar Graphs

Entry ID: 2b8c15d516fe **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 00:37:05 UTC

1. Conjecture

Field A (mathematical branch): Quadratic Residue Theory **Field B** (complexity object): Communication Complexity

Statement:

For any given planar graph G with n vertices, the number of distinct quadratic residues modulo a prime p that are represented by edge weights in G is linearly correlated with the communication complexity growth rate of the associated communication game, such that $|Q(G)| = \Theta(g(n))$ for some function $g(n)$.

Rationale (proposer's reasoning):

Quadratic residue theory provides an algebraic structure that could potentially encode combinatorial properties of graphs. Communication complexity measures how efficiently two parties can communicate to compute a function over their joint input. By examining the relationship between quadratic residues in graph edge weights and communication complexity, this conjecture aims to expose an algebraic insight into the computational aspects of communication.

Taxonomy category: Quadratic Residue Theory × Communication Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c49c02c34ab12075

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a planar graph G with n vertices, if the Pearson correlation coefficient between $|Q(G)|$ and $g(n)$ exceeds 0.7 for at least 24 out of 30 seeds, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quadratic residue theory" AND "communication complexity" AND planar graph - "edge weights" IN planar graph AND quadratic residues modulo prime p - "communication game complexity growth rate" correlated with $|Q(G)| \theta(g(n))$

Top relevant hits considered: - [http://arxiv.org/abs/2406.11937v3] Using graph neural networks to reconstruct charged pion showers in the CMS High Granularity Calorimeter - [http://arxiv.org/abs/1408.5939v2] Planar Induced Subgraphs of Sparse Graphs - [http://arxiv.org/abs/2410.135] Observation of a rare beta decay of the charmed baryon with a Graph Neural Network - [http://arxiv.org/abs/1808.08676v3] Constraining the p-mode-g-mode tidal instability with GW170817 - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_planar(n):
        if n < 3:
            return True
        for i in range(1, n):
            for j in range(i + 1, n):
                for k in range(j + 1, n):
                    if (i * j + j * k + k * i) % (n - 1) == 0:
                        return False
        return True

    def generate_planar_graph(n):
        while not is_planar(n):
            edges = set()
            for _ in range(3 * n - 6):
                u, v = random.sample(range(n), 2)
```

```

        if (u, v) not in edges and (v, u) not in edges:
            edges.add((u, v))
    return list(edges)

def quadratic_residues(p):
    residues = set()
    for i in range(1, p):
        residues.add(i * i % p)
    return residues

def communication_complexity(n):
    # Placeholder function; actual implementation depends on the game
    return n ** 2

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    graph = generate_planar_graph(n)
    p = random.randint(2, n - 1)
    residues = quadratic_residues(p)
    Q_G = len(residues.intersection({w % p for u, v in graph for w in range(n)}))
    g_n = communication_complexity(n)
    results.append((Q_G, g_n))

if not results:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": 0.0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

Q_G_values = [Q for Q, _ in results]
g_n_values = [g for _, g in results]

mean_Q_G = sum(Q_G_values) / len(Q_G_values)
mean_g_n = sum(g_n_values) / len(g_n_values)

covariance = sum((Q_G - mean_Q_G) * (g_n - mean_g_n) for Q_G, g_n in results) / len(results)
variance_Q_G = sum((Q_G - mean_Q_G) ** 2 for Q_G in Q_G_values) / len(Q_G_values)
variance_g_n = sum((g_n - mean_g_n) ** 2 for g_n in g_n_values) / len(g_n_values)

if variance_Q_G == 0 or variance_g_n == 0:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": 0.0,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "variance_zero"
    }

pearson_corr = covariance / (math.sqrt(variance_Q_G) * math.sqrt(variance_g_n))

```

```

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": pearson_corr,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": abs(pearson_corr) > 0.7,
    "counterexample": "" if abs(pearson_corr) > 0.7 else f"correlation={pearson_corr}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not results:
        print("RESULT: INCONCLUSIVE no_trials")
        sys.exit(0)

    mean_corr = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_corr} std=0.0 support_fraction={support_fraction}")
    elif any(r["counterexample"] == "mapping_undefined" for r in results):
        print("RESULT: INCONCLUSIVE mapping_undefined")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='correlation_too_low' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7b9c140b.py", line 108, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7b9c140b.py", line 57, in run_trial
    Q_G = len(residues.intersection({w % p for u, v in graph for w in range(n)}))
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: 'NoneType' object is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete the required calculations for the Pearson correlation coefficient and the number of significant seeds. | next: Review the code for potential bugs and ensure that it can handle all possible cases without crashing. Then re-run the test with a larger sample size to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16064
2	propose	ollama_remote	glm4:latest	0	0	13785
3	preregistration	ollama_remote	glm4:latest	0	0	28553
4	novelty	ollama_remote	glm4:latest	0	0	22416
5	novelty	ollama_remote	glm4:latest	0	0	15511
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19124
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14233
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27974
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16375
10	judge	ollama_remote	glm4:latest	0	0	11756

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 185791 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/2b8c15d516fe.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/2b8c15d516fe.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/2b8c15d516fe.tar.gz (if generated)